

From Property-Based Tests to Formal Proofs: A Benchmark for Lean 4 Specification Generation

Abstract—We present a benchmark for evaluating AI models on formal verification tasks, extending the RealPBT dataset of real-world Python property-based tests (PBTs) into Lean 4 specifications with sorry placeholders. Translating PBTs into formal specifications is challenging: it requires modelling Python library semantics in Lean, bridging the gap between input/output testing and structural proofs, and handling the inherent difficulties of dependently-typed programming. We describe a three-agent LLM pipeline for transpiling PBT suites into Lean specifications, evaluate coverage and quality metrics, and provide baselines for proof completion using several automated and model-based approaches. Our benchmark aims to drive progress on the underexplored problem of AI-assisted formal verification of real-world software.

Index Terms—Formal verification, Property-based testing, Lean 4, Benchmark, LLM

I. INTRODUCTION

Several AI safety proposals are premised on AI-driven *formal verification* (FV); this is a core hypothesis of the ARIA Safeguarding AI program and the related Guaranteed Safe AI agenda [1]. If FV is to play this role, it must scale to the complexities of real-world engineered systems, but today we have little evidence whether such capabilities are possible. Relevant benchmarks are mostly either small datasets crafted by hand [2], larger datasets extracted from expert-driven verification projects [3], or are focused on advanced mathematics rather than engineering [4]. None of these are likely to be representative of real-world engineering tasks.

We generate a new benchmark, FV-Spec, from public uses of *property-based testing* (PBT). While FV itself is rarely used in software development, PBT is a ‘close sibling’ technology which has seen significant adoption. In both FV and PBT, engineers write *specifications*—logical properties that the software must obey. For example, we might require that a Python function `double(x)` always returns `2*x`. The difference lies in how this property is checked. In FV, the property is *proved* using mathematical techniques—this results in almost-perfect confidence, but requires *proof engineering* by highly expert teams [5]. In PBT, the code is randomly tested—e.g. for random values of `x`. This is a push-button process with no proof engineering.

PBT is typically much cheaper than formal verification, and as a result has seen wider adoption. We build our benchmark from RealPBT, a dataset of 54k permissively-licensed PBTs written in Python’s Hypothesis framework [6]. Each PBT in our dataset can be seen as a small theorem about a piece of Python code. To make it possible for an AI to prove or refute these theorems, we translate both code and PBTs to the theorem prover Lean, which is becoming the de-facto standard in AI formal verification. This approach has been pioneered by Dougherty and Mehta on the FVAPPS

verification benchmark [7]. In FVAPPS, source programs were taken from the APPS benchmark by Hendrycks et al [8], translated to PBTs, then lifted to theorems. We apply the same process at scale to PBTs found in the wild.

We use AI-driven translation to generate Lean programs and theorems at scale. LLMs are quite good at program translation, and FVAPPS has shown that this approach is practical for constructing Lean theorems. The result is tens of thousands of verification challenges, each consisting of (1) a program and (2) a desired specification, both written in Lean. Unlike existing FV benchmarks, each problem corresponds to properties of real-world software written by engineers with no formal verification experience.

II. TASK DEFINITION

Each problem in the FV-Spec benchmark consists of three components: (1) the original Python property-based test, (2) a Lean theorem corresponding to the PBT, with a sorry placeholder where the proof should go, and (3) a Lean implementation of the functions referenced in the theorem. The AI’s task is to complete the proof by replacing sorry with a valid Lean proof term. Each problem also includes a URL pointing to the original source code and license information.

A. Running Example

Consider a Hypothesis test asserting the Pythagorean trigonometric identity, $\sin^2(x) + \cos^2(x) = 1$:

```
from hypothesis import given, strategies

@given(x=strategies.floats(
    min_value=-sys.float_info.max / 4,
    max_value=sys.float_info.max / 4,
    allow_infinity=False))
def test_property(x):
    assert old.sine(x) ** 2 + old.cosine(x) ** 2 ==
pytest.approx(1)
```

Our pipeline translates this into a Lean implementation:

```
namespace Fvspec.Impl

def to_radians (degrees : Float) : Float :=
    degrees * Float.pi / 180.0

def sine (θ : Float) : Float :=
    Float.sin (to_radians θ)
```

```
end Fvspec.Impl
```

And a corresponding Lean specification:

```
import Fvspec.Impl
open Fvspec.Impl

def cosine (θ : Float) : Float := sorry
```

`theorem sine_cosine_pythagorean (x : Float) :`

`(sine x) * (sine x) + (cosine x) * (cosine x) = 1 := sorry`

The AI must fill in both sorry placeholders with valid Lean proof terms.

B. PBT vs. Proof

In PBT, a property is checked by randomly generating inputs and testing them—the process is push-button but provides only statistical confidence. In FV, the property is *proved* using mathematical techniques, providing near-perfect confidence but requiring proof engineering.

Translating a PBT into a formal specification thus involves a shift from input/output testing to proofs over the structure of the program. Our pipeline filters nonsensical theorems, but any discrepancies that remain will not affect the usefulness of the dataset. Because the Lean theorem prover provides a ‘perfect’ oracle, every theorem we generate is a useful challenge problem, even if it differs from the original PBT. In fact, some PBTs in our input dataset will be naturally false due to undetected edge cases.

C. Challenges

Formal verification is unusually difficult to benchmark for two reasons. First, just as with advanced mathematics, formal verification tasks are highly varied, and constructing interesting problems requires considerable expertise. FV tasks also vary in difficulty—they can be as shallow as checking the type-safety of simple functions, or as deep as proving an unsolved mathematical theorem such as the Collatz Conjecture. Our benchmark avoids the need to select and hand-build challenge problems by translating naturally occurring PBT problems written by real-world engineers.

Second, formal verification is little used outside academia. As a result, there are few natural datasets, and those that exist are fragmented between multiple FV tools (e.g. SAW/Cryptol for cryptographic systems, ACL2/Coq for traditional software verification, Lean for mathematics, SPIN/TLA+ for protocol analysis). Public FV examples are often pedagogical exercises for which the proof exists in many different forms in the training set. Our benchmark solves this problem by translating from a ‘close sibling’ domain, PBT, where many more datapoints exist. As a result, most theorems in our benchmark will have never been verified by anyone.

Beyond these structural challenges, AI-driven translation at scale introduces practical difficulties. Each failed pipeline run is expensive—a single sample may require multiple LLM calls with iterative repair, and a bug in the pipeline infrastructure can invalidate an entire batch. Quality improvements must therefore be targeted: we identify specific failure modes in the output (e.g. type restrictions, missing dependencies, trivial stub theorems) and address them incrementally rather than re-running the full dataset. We initially considered extending the benchmark to TypeScript PBTs, but found that the PBT ecosystem in TypeScript is far less mature, and the additional coverage would not justify the engineering effort.

III. PIPELINE

Our pipeline is modelled on the one used by Dougherty and Mehta for FVAPPS [7]. We begin with the RealPBT dataset, which has already been cleaned and filtered.

Pipeline -

- Pick a sample from RealPBT
- Load a prompt that says “these are the functions under test”
- Says this is the spec
- Dispatch to two agents
 - function Under test formalizer
 - Instructed to be completely computable
 - No sorries at all
 - PBT formalizer
 - instructed to sorry everywhere
- Agent loop (both agents)
 - Call Lean-LSP-MCP
 - check whether the formalized code type-checks in Lean
 - Call the agent
- Loop exit
 - Static evaluation via LLM judges
 - Difficulty evaluation - measure 1-10 difficulty
 - Faithfulness - does the code / test actually match the original

(eg structural faithfulness: { “parameter_coverage”: 0, “type_correspondence”: 1, “strategy_coverage”: 1, “assertion_coverage”: 1, “dependency_coverage”: 1, “assertion_theorem_difference”: -1, “overall”: 0.75 })

For each sample drawn from the RealPBT dataset, we first use tree-sitter to extract the Python source code of the function under test (FUT) and any functions it depends on. This *function discovery* step succeeds for approximately 92% of samples; when it fails, we generate a stub implementation with generic type parameters to avoid trivial theorems.

The extracted code is then processed by three LLM agents. The *implementation agent* runs first, translating the FUT and its dependencies into fully computable Lean definitions—no sorry placeholders are permitted, so all code must typecheck and evaluate. Dependencies are formalized separately and merged with deduplication into a single Impl.lean file. If the implementation agent succeeds, we extract the Lean type signatures of all formalized functions and pass them as context to the next stage. If it fails (i.e. it exhausts its retry budget without producing typechecking code), the downstream agents still run, but without a verified implementation to build on.

Next, the *specification agent* and *units agent* run in parallel. The specification agent translates the Hypothesis tests into Lean theorems that import the formalized implementation, using sorry for all proof obligations. The units agent translates any available Python unit tests (drawn from a database of 6.3M tests) into LSpec test cases. The units agent’s output is included in the benchmark for additional validation but does not gate the specification.

All agents operate in an iterative repair loop. At each step, the agent’s output is checked against the Lean compiler via the Lean LSP (accessed through an MCP tool server). If the code does not typecheck, the compiler diagnostics are fed back to the agent, which attempts a revised translation. This loop continues until the code typechecks or a retry budget is exhausted. Final compilation is validated with a full lake build invocation.

The generated specifications undergo two forms of quality assessment. *Structural faithfulness* is computed programmatically by analyzing the source code: the metric decomposes into sub-scores for parameter coverage, type correspondence, strategy coverage, assertion coverage, and dependency coverage, producing a weighted overall score (e.g. 0.75 out of 1.0). *Difficulty* is assessed by an LLM judge (Claude Haiku), which rates each problem on a 1–10 scale estimating the proof effort required.

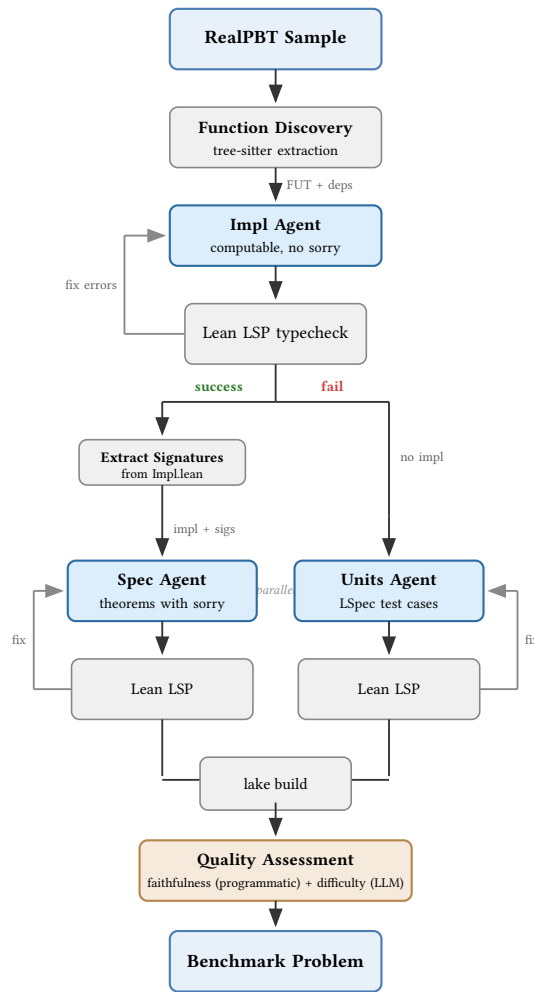


Fig. 1. Overview of the FV-Spec generation pipeline. Function discovery extracts the Python source, then the implementation agent formalizes it to computable Lean. On success, its output and extracted type signatures feed into the specification and units agents, which run in parallel. On failure, the downstream agents proceed without a verified implementation. All agents iteratively repair their output against the Lean LSP. The final result is scored for structural faithfulness (programmatic) and difficulty (LLM judge).

A. LLM-Based Transpilation

Our three-agent transpilation pipeline operates as follows:

- Identify target property-based tests on Github covered by permissive licenses.
- Using a mix of syntactic translation and code-focused LLM, translate to Lean code and properties. As in FVAPPS, we iteratively repair partially-successful translations over multiple steps.
- Use Lean’s property-based testing framework Plausible [9] alongside the code-focused LLM to filter poor-quality translations, and target them for repair.

Based on FVAPPS, we are confident that smaller problems and code can be easily transpiled using an LLM. Larger examples may fail or generate poor-quality translations. In such cases, we fall back to syntactic translation, which we expect to produce less understandable but more predictable results, and then use LLMs for targeted quality improvement.

B. Cleanup and Quality Evaluation

In some cases, transpilation may generate less useful specification/program pairs. For example, a common failure in FVAPPS was type restriction in specifications, such as translating a signed integer into an unsigned natural. We investigate pairwise testing between original and translated programs to detect such discrepancies. When function discovery fails (8% of samples), the implementation agent receives no source code and must generate a stub with generic type parameters; theorems built on such stubs are flagged as lower-quality and may be filtered. We also found that LLM-generated implementations sometimes introduce `\#eval` statements that pass in isolation but cause build failures when combined with the specification; these are detected and stripped in postproduction. We filter and review challenge problems (akin to the SWE-bench Verified effort [10]), build clear and usable documentation, and develop a website with leaderboards.

C. Metrics

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

IV. RESULTS

A. Experimental Setup

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

B. Coverage

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus

animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

C. Quality

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

D. Baselines

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

E. Difficulty Stratification

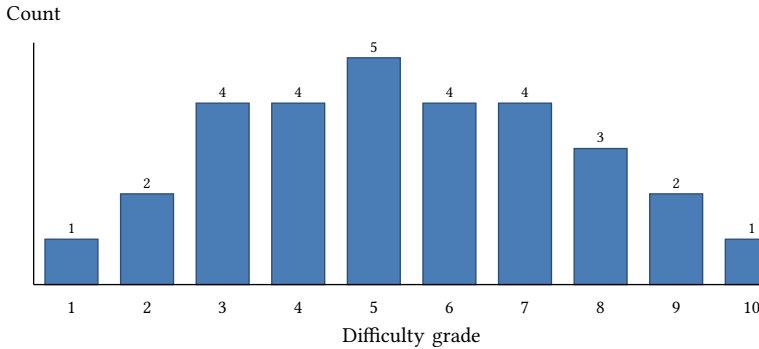


Fig. 2. Distribution of Haiku-assessed difficulty grades across the 30 benchmark tasks.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

V. THREATS TO VALIDITY

We note that the theorems we construct *may be false*—this is desirable because the AI has the opportunity to contradict the theorem (and perhaps identify a counterexample). Because the Lean theorem prover is a so-called *foundational* tool, once a theorem is verified or contradicted, we have near-perfect confidence the answer is correct. However, because our specifications are translated rather than hand-written, some may not faithfully represent the original PBT. Our structural faithfulness metric provides an objective measure of translation quality, but it is a heuristic: a high score does not guarantee semantic equivalence, and a low score does not necessarily indicate a useless problem.

Our benchmark does not include human baselines. We expect the dataset to exhibit a wide range of difficulty, including many problems that are trivial for current AIs, and many harder problems, including some insurmountable even to world-expert human teams. Without human expert

performance data, we cannot directly calibrate this difficulty distribution.

The pipeline’s reliance on LLM-driven translation means that benchmark quality is bounded by the capabilities of the translation model. Samples where function discovery failed (8%) produce stub-based theorems that may be less representative. We mitigate this through postproduction filtering and the structural faithfulness metric, but some lower-quality samples may remain.

VI. RELATED WORK

Our work extends FVAPPS by Dougherty and Mehta [7], which pioneered AI-driven translation of coding problems into Lean theorems. In FVAPPS, source programs were taken from the APPS benchmark [8], translated to PBTs, then lifted to theorems. We apply the same process at scale to PBTs found in the wild, rather than synthetic coding challenges.

Lean verification benchmarks. A growing number of benchmarks target Lean specifically. miniCodeProps [11] provides 177 program specifications translated from TIP (Tons of Inductive Programs), finding that current LLM provers fail on medium and hard properties. CLEVER [12] offers 161 curated problems sourced from HumanEval, testing end-to-end verified code generation. Verina [13] covers all three foundational tasks—code generation, specification generation, and proof generation—across 189 manually curated coding tasks. VeriBench [14] translates Python code with documentation into Lean implementations, specifications, and unit tests (113 tasks). The Vericoding Benchmark [15] supports Lean, Rust/Verus, and Dafny, reporting success rates from 27% (Lean) to 82% (Dafny). VeriEquivBench [16] uses LeetCode transformations to evaluate verifiable agents on novel data without contamination. These benchmarks are valuable but share a common limitation: their specifications derive from synthetic coding problems or pedagogical exercises, not from properties written by practicing engineers.

Other verification tool benchmarks. DafnyBench [2] provides Dafny verification tasks. VerusBench [17] collects 150 proof tasks spanning Dafny, SV-COMP, and Verus. SV-COMP [18] is the standard competition for software verifiers, with over 33,000 C tasks, but targets automated verification rather than interactive theorem proving. InvBench [19] focuses specifically on loop invariant synthesis. Chakraborty et al. [3] explore neural synthesis for SMT-assisted proof-oriented programming in F*.

Interactive theorem proving benchmarks. LeanDojo Benchmark 4 [20] provides 122,517 theorems from Mathlib4. CoqStoq [21] collects 196,929 theorems from 2,226 GitHub Coq repositories. These are primarily mathematical rather than software-oriented, and their theorems exist in the training data of most frontier models.

Mathematics benchmarks. FrontierMath [4] evaluates advanced mathematical reasoning. While formal verification and mathematics share proof infrastructure, the skills required differ substantially: software verification demands modelling of program semantics, library APIs, and computational behavior, rather than abstract mathematical reasoning.

FV-Spec differs from existing benchmarks in three ways. First, our specifications originate from real-world property-based tests written by practicing engineers, not synthetic problems or pedagogical exercises. Second, the source PBTs were never formally verified, so most theorems in our benchmark will not appear in any model’s training data. Third, we operate at a larger scale than existing Lean verification benchmarks, with thousands rather than hundreds of problems. Our work is motivated by the broader context of AI safety proposals premised on AI-driven formal verification, including the Guaranteed Safe AI agenda [1].

VII. CONCLUSION

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

A. Future Work

Several extensions are possible:

- Add examples from other languages with significant usage of PBT, e.g. Haskell/QuickCheck, C++/Rapid-Check.
- Translate our benchmark to other theorem proving systems, e.g. Coq, Isabelle, F*, etc.
- More advanced website features, such as prediction markets about when the benchmark will be saturated, and a pipeline for submitting code to the leaderboard which we then persist into a training or SFT dataset.

Our benchmark will also be useful for developing new techniques in program repair and synthesis. If a property is disproven, this creates a new interesting challenge, namely to synthesize a patch to the original function such that the theorem now holds. We hypothesize that many such bugs will be particularly subtle and interesting edge cases since, by definition, in most cases they were not detected by the original developer’s PBT harness.

REFERENCES

- [1] D. Dalrymple *et al.*, “Towards Guaranteed Safe AI: A Framework for Ensuring Robust and Reliable AI Systems,” *arXiv preprint arXiv:2405.06624*, 2024.
- [2] C. Loughridge *et al.*, “DafnyBench: A Benchmark for Formal Software Verification,” *arXiv preprint arXiv:2406.08467*, 2024.
- [3] S. Chakraborty *et al.*, “Towards Neural Synthesis for SMT-Assisted Proof-Oriented Programming,” in *Proceedings of the 47th International Conference on Software Engineering (ICSE)*, 2025.
- [4] E. Glazer *et al.*, “FrontierMath: A Benchmark for Evaluating Advanced Mathematical Reasoning in AI,” *arXiv preprint arXiv:2411.04872*, 2024.
- [5] M. Dodds, “Formally Verifying Industry Cryptography,” *IEEE Security & Privacy*, vol. 20, no. 3, pp. 65–70, 2022, doi: 10.1109/MSEC.2022.3153035.
- [6] D. R. MacIver, Z. Hatfield-Dodds, and many other contributors, “Hypothesis: A New Approach to Property-Based Testing,” *Journal of Open Source Software*, vol. 4, no. 43, p. 1891, 2019, doi: 10.21105/joss.01891.
- [7] Q. Dougherty and R. Mehta, “Proving the Coding Interview: A Benchmark for Formally Verified Code Generation,” in *Proceedings of the LLM4Code Workshop at the 47th International Conference on Software Engineering (ICSE)*, Ottawa, Ontario, Canada, 2025. doi: 10.48550/arXiv.2502.05714.
- [8] D. Hendrycks *et al.*, “Measuring Coding Challenge Competence With APPS,” in *Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS)*, 2021.
- [9] K. Morrison, C. Roux, H. Böving, S. Ullrich, and T. Levy, “Plausible: A Property Testing Framework for Lean 4.” [Online]. Available: <https://github.com/leanprover-community/plausible>
- [10] C. E. Jimenez *et al.*, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?,” in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.
- [11] D. Brandfonbrener, L. Beurerkellner, D. Romero, N. Amin, and M. Vechev, “miniCodeProps: a Minimal Benchmark for Proving Code Properties,” *arXiv preprint arXiv:2406.11915*, 2024.
- [12] M. Materzok, W. Szulc, M. Tabiś, J. Kozak, and B. Piotrowski, “CLEVER: A Benchmark for Closed-Loop End-to-End Formally Verified Code Generation,” *arXiv preprint arXiv:2505.13938*, 2025.
- [13] W.-D. Wu, K. Wang, R. Mangal, and I. Dillig, “Verina: Benchmarking Formally Verified Code Generation,” *arXiv preprint arXiv:2505.23135*, 2025.
- [14] K. Stechly and others, “VeriBench: Benchmarking End-to-End Formal Verification,” in *AI4Math Workshop at ICML*, 2025. [Online]. Available: <https://openreview.net/forum?id=rWkGFmnSNl>
- [15] K. Stechly and others, “The Vericoding Benchmark,” *arXiv preprint arXiv:2509.22908*, 2025.
- [16] X. Ma and others, “VeriEquivBench: Evaluating Verifiable Agents on Novel Data Without Contamination,” *arXiv preprint arXiv:2510.06296*, 2025.
- [17] H. Chen and others, “VerusBench: A Benchmark for Automated Verification of Rust with Verus,” *arXiv preprint arXiv:2409.13082*, 2024.
- [18] D. Beyer, “State of the Art in Software Verification and Witness Validation: SV-COMP 2025,” in *Proceedings of the 31st International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2025.
- [19] T.-Y. Wu and others, “InvBench: A Benchmark for Loop Invariant Synthesis,” *arXiv preprint arXiv:2509.21629*, 2025.
- [20] K. Yang *et al.*, “LeanDojo: Theorem Proving with Retrieval-Augmented Language Models,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [21] G. Karas, A. Joshi, S. Vaughn, S. Bhat, and Q. Dougherty, “CoqS-toq: Proving Coq Theorems from the GitHub Wild,” *arXiv preprint arXiv:2412.14063*, 2024.